Rapport de test automatisé JUnit

Université Polytechnique
HAUTS-DE-FRANCE

Rapport de test automatisé JUnit S2.01

BICHALL TATY Lordevy Billy
BOUCHNAK Kaouther
KHEMIRI Chahd
SORGHO Reine Pélagie
TOVIEKOU Sosthène

BUT 1 INFORMATIQUE GROUPE A2
ANNÉE-UNIVERSITAIRE : 2025-2026

Plan de présentation

Université Polytechnique HAUTS-DE-FRANCE

Plan de présentation

1. Introduction

- Contexte du projet (SAÉ 2.01)
- Objectifs du rapport de tests automatisés
- Présentation de la classe principale testée : Robot

2. Stratégie et Explication du Choix des Tests

- Approche par tests Structurels (Boîte Blanche)
- Justification du choix de la couverture des branches
- Approche par tests Fonctionnels (Boîte Noire)
- Technique de partitionnement en classes d'équivalence

3. Tableaux des Confections de Tests

- Tests Boîte Blanche : Méthode avancer
- identification des chemins (après avoir fait le graph de flot)
- Tableau de test des branches (Directions, Secteurs, Obstacles)
- Tests Boîte Noire : Méthode extraire
- Matrice des valeurs limites (Quantités de minerais et validité)
- Tests Boîte Blanche : Méthode deposer
- Tableau de test des chemins (Composantes Mine/Entrepôt et types de minerais)

4. Exécution des Tests et Captures d'Écran

- Protocole de lancement des tests automatisés
- Captures d'écran des résultats d'exécution (Barre verte JUnit)
- Rapport de couverture du code (Code Coverage)

5. Bilan et Conclusion

- Analyse des résultats (Suivi des anomalies détectées)
- Fiabilité et robustesse du code de la classe Robot
- Conclusion générale

Introduction

Introduction:

Contexte du projet (SAÉ 2.01)

Dans le cadre de la SAÉ 2.01 du BUT 1 Informatique à l'IUT de Valenciennes (Campus de Maubeuge), notre équipe du Groupe A2 a développé une application Java de simulation de robots mineurs. Réalisé avec IntelliJ IDEA (JDK 21) et Maven, ce projet repose sur la gestion d'un monde matriciel de 10*10 secteurs où des robots doivent extraire des minerais (or ou nickel) et les transporter vers des entrepôts dédiés.

Objectifs du rapport de tests automatisés (Livrable 1.04)

Conformément aux exigences du livrable 1.04 de la SAÉ 2.01, ce document constitue notre rapport de tests unitaires automatisés.La mise en place d'une suite de tests rigoureuse est essentielle pour valider le comportement de l'application vis-à-vis des règles métiers définies par le sujet (gestion des obstacles, limites de capacité, types de minerais).

Ce rapport a pour double objectif:

1. De formaliser notre jeu de tests à travers des tableaux de spécification (boîte noire et boîte blanche) conformes aux critères de conception étudiés dans la ressource de tests (R2.03).
2. De présenter et d'analyser le rapport d'exécution de ces tests au sein de notre environnement de développement (validé par les captures des résultats d'exécution JUnit).

Périmètre des tests :

avancer : Boîte Blanche (Tests Structurels) : Cette méthode gère les déplacements du robot dans la grille de 10*10 secteurs selon les quatre points cardinaux. Les tests visent à couvrir l'ensemble des branches du code : validation du déplacement nominal, détection de collision avec un autre robot, interdiction de pénétrer sur un plan d'eau (XX), et gestion des limites de la matrice.

extraire : Boîte Noire (Tests Fonctionnels) : Destinée à la collecte de ressources, cette méthode pilote le prélèvement de minerais en faisant interagir le robot avec les classes de gestion des **Mines**. Validés par l'analyse des valeurs limites, les tests vérifient le respect de la capacité d'extraction (1 à 3 minerais) , la gestion d'une mine épuisée , et la stricte adéquation entre la spécialité du robot et le type de la mine (or ou nickel).

deposer : Boîte Blanche (Tests Structurels) : Cette méthode assure le déchargement des minerais transportés. Les tests analysent les différents chemins d'exécution pour garantir

que le dépôt ne réussit que si le robot se trouve sur le secteur de l'entrepôt correspondant spécifiquement à sa spécialité (or ou nickel).

Stratégie et Explication du Choix des Tests

Stratégie et Explication du Choix des Tests :

Pour garantir la qualité, la robustesse et la non-régression de l'application, nous avons mis en place une stratégie de tests unitaires automatisés avec JUnit 5. Nous avons combiné deux approches méthodologiques complémentaires: l'approche structurelle (boîte blanche) et l'approche fonctionnelle (boîte noire).

Approche par tests Structurels (Boîte Blanche)

L'objectif est de s'assurer que chaque transition et chaque branche conditionnelle (if, else, switch) du code a été exécutée au moins une fois. Pour la méthode avancer, cela permet de valider tous les chemins menant à un déplacement réussi ou à un échec selon les obstacles rencontrés.

Approche par Tests Fonctionnels (Boîte Noire)

Contrairement à la boîte blanche, l'approche boîte noire se focalise uniquement sur les spécifications fonctionnelles et les règles métiers du sujet, sans se soucier du code interne. Nous avons choisi cette méthode pour valider de manière stricte la méthode extraire.

- **Partitionnement en classes d'équivalence :** Nous avons divisé les données d'entrée en sous-ensembles de valeurs traitées de la même manière par l'application (les valeurs valides et les valeurs invalides).
- **Analyse des valeurs limites :** C'est le critère principal retenu pour la méthode extraire. Les erreurs logiques apparaissent souvent aux frontières des conditions, nous avons testé le comportement du système aux valeurs pivots (ex: capacités minimales, maximales et dépassements de capacité d'extraction de 1 à 3 minerais décrits dans le sujet).

Tableau Boite blanche avancer

Tableau Boite blanche class Robot : méthode avancer

Code de la méthode avancer

```java
public boolean avancer(Directions dir, Secteur[][] grille) {  18 usages  Sosthene +2

    if (secteur == null) {
        System.out.println("Action invalide : le robot n'est pas placé sur le plateau.");
        return false;
    }

    int ligne   = secteur.getLigne();
    int colonne = secteur.getColonne();

    int nouvelleLigne   = ligne;
    int nouvelleColonne = colonne;

    if (dir == Directions.NORD) {
        nouvelleLigne--;
    } else if (dir == Directions.SUD) {
        nouvelleLigne++;
    } else if (dir == Directions.EST) {
        nouvelleColonne++;
    } else if (dir == Directions.OUEST) {
        nouvelleColonne--;
    }

    int maxLignes = grille.length;
    int maxCols   = grille[0].length;

    if (nouvelleLigne < 0 || nouvelleLigne >= maxLignes
            || nouvelleColonne < 0 || nouvelleColonne >= maxCols) {
        System.out.println("Action invalide : hors du monde !");
        return false;
    }

    Secteur cible = grille[nouvelleLigne][nouvelleColonne];

    if (cible.EstEau()) {
        System.out.println("Action invalide : plan d'eau !");
        return false;
    }

    if (cible.getRobot() != null) {
        System.out.println("Action invalide : secteur déjà occupé par un robot !");
        return false;
    }

    secteur.setRobot(null);
    cible.setRobot(this);
    this.secteur = cible;

    System.out.printf("Robot %d avance vers %s%n", numero, dir);
    return true;
}
```

identification des chemins après avoir fais le graph de flot

Test :Couverture Branche

Chemin	Direction	nouvelle colonne	nouvelle Ligne	Secteur	Résultat Attendue
(24-26,26-28,28-74)	-	-	-	null	False
(24–26,26–31,31–37,37–50,50–56,56–58,58–60,60–74)	Nord	1	1	eau	False(prob eau)
(24–26,26–31,31–37,37–50,50–56,56–58,58–63,63–74)	Nord	5	6	vide	True
(24–26,26–31,31–37,37–50,50–56,56–58,58–63,63–65,65–74)	Nord	2	5	robot	False(prob robot)
(24–26,26–31,31–37,37–50,50–53,53–74)	Nord	0	5	!null	False(case invalide)
(24–26,26–31,31–37,37–39,39–41,41–43,43–74)	null	-	-	!null	prob case pas vide(robot)
(24–26,26–31,31–37,37–39,39–50,50–56,56–58,58–63,63–65,65–74)	Sud	5	3	Robot	False
(24–26,26–31,31–37,37–39,39–41,41-50,50–56,56–58,58–63,63–65,65–74)	Est	5	8	Robot	False
(24–26,26–31,31–37,37–39,39–41,41–43,43–50,50–53,53–74)	Ouest	11	-5	!null	False

Tests junits

```
package TEST;
import org.example.Model.Directions;
import org.example.Model.Robot;
import static org.junit.jupiter.api.Assertions.*;

import org.example.Model.Secteur;
import org.example.Model.TypeMinerai;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;


public class TestAvancerRobot
{
   protected Secteur[][] grille;

   @BeforeEach
   void setUp() throws Exception {
       grille = new Secteur[10][10];
       for (int l = 0; l < 10; l++) {
           for (int c = 0; c < 10; c++) {
               grille[l][c] = new Secteur(l, c, false);
           }
       }
   }
   @AfterEach
   void tearDown() throws Exception
   {
       grille = null;
   }
   @Test
   void test1()
   {
       Robot robot = new Robot(1,null,TypeMinerai.OR,4,2);

assertEquals(false,robot.avancer(Directions.NORD,grille));
   }
   @Test
   void test2()
   {
       Secteur secteurDepart = grille[2][1];
       Robot robot = new
Robot(1,secteurDepart,TypeMinerai.OR,4,2);
```

```
        secteurDepart.setRobot(robot);
        grille[1][1].setEstEau(true);
        assertEquals(false, robot.avancer(Directions.NORD,
grille));

    }
    @Test
    void test3()
    {
        Secteur secteurDepart = grille[5][6];
        Robot robot = new
Robot(1,secteurDepart,TypeMinerai.OR,4,2);
        secteurDepart.setRobot(robot);
        assertEquals(true, robot.avancer(Directions.NORD,
grille));
    }
    @Test
    void test4()
    {
        Secteur secteurDepart = grille[2][5];
        Robot robot = new
Robot(1,secteurDepart,TypeMinerai.OR,4,2);
        secteurDepart.setRobot(robot);
        Secteur secteurCible = grille[1][5];
        Robot robot2 = new Robot(2, secteurCible,
TypeMinerai.OR, 4, 2);
        secteurCible.setRobot(robot2);
        assertEquals(false, robot.avancer(Directions.NORD,
grille));
    }
    @Test
    void test5()
    {
        Secteur secteurDepart = grille[0][5];
        Robot robot = new
Robot(1,secteurDepart,TypeMinerai.OR,4,2);
        secteurDepart.setRobot(robot);
        assertEquals(false, robot.avancer(Directions.NORD,
grille));
    }
    @Test
    void test6()
    {
        Secteur secteurDepart = grille[1][1];
```

```
        Robot robot = new Robot(1, secteurDepart,
TypeMinerai.OR, 4, 2);
        secteurDepart.setRobot(robot);
        assertEquals(false, robot.avancer(null, grille));
    }
    @Test
    void test7()
    {
        Secteur secteurDepart = grille[1][5];
        Robot robot = new
Robot(1,secteurDepart,TypeMinerai.OR,4,2);
        secteurDepart.setRobot(robot);
        Secteur secteurCible = grille[2][5];
        Robot robot2 = new Robot(2, secteurCible,
TypeMinerai.OR, 4, 2);
        secteurCible.setRobot(robot2);
        assertEquals(false, robot.avancer(Directions.SUD,
grille));
    }
    @Test
    void test8()
    {
        Secteur secteurDepart = grille[1][4];
        Robot robot = new
Robot(1,secteurDepart,TypeMinerai.OR,4,2);
        secteurDepart.setRobot(robot);
        Secteur secteurCible = grille[1][5];
        Robot robot2 = new Robot(2, secteurCible,
TypeMinerai.OR, 4, 2);
        secteurCible.setRobot(robot2);
        assertEquals(false, robot.avancer(Directions.EST,
grille));
    }
    @Test
    void test9()
    {
        Secteur secteurDepart = grille[1][0];
        Robot robot = new
Robot(1,secteurDepart,TypeMinerai.OR,4,2);
        secteurDepart.setRobot(robot);
        assertEquals(false, robot.avancer(Directions.OUEST,
grille));
    }
```

```
}
```

Tableaux des Confections de Tests

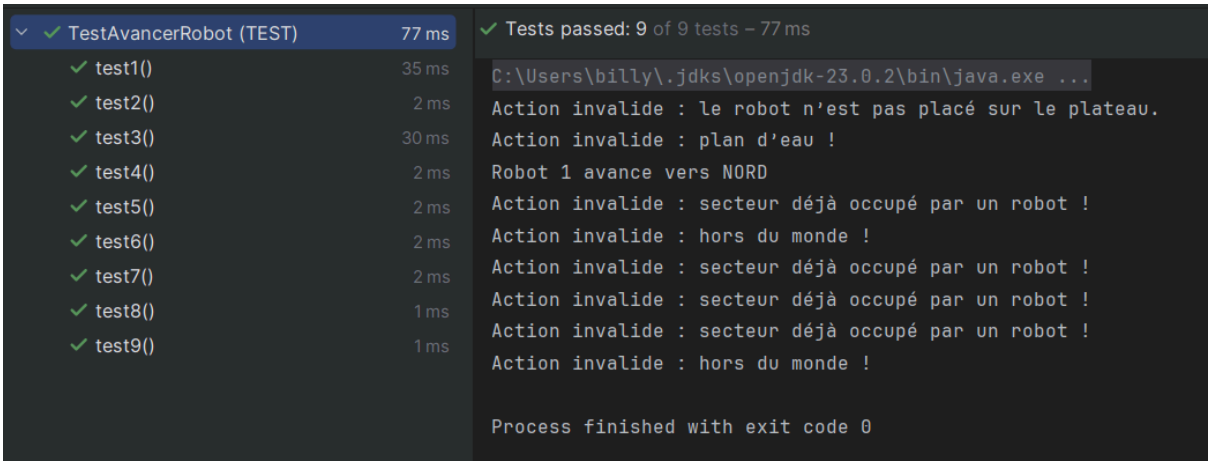


Tableau de test : boite noire

Université Polytechnique HAUTS-DE-FRANCE

Tableau de test classe Mine: boite noire (méthode extraire)

Code de la méthode extraire:

```java
public int extraire(int qte) {  21 usages   reinesorgho +1
    if (qte <= 0 || qte > 3) {
        throw new IllegalArgumentException("Erreur : La quantité demandée doit être comprise entre 1 et 3.");
    }
    if (estVide()) return 0;
    int quantiteExtraite = Math.min(qte, nbMinerais);
    nbMinerais -= quantiteExtraite;
    return quantiteExtraite;
}
```

Valeurs limites :

nbMn	qte	valeur attendue(qt extraire)	valide/invalide
-1	tout	message d'erreur	invalide
tout	-1	message d'erreur	invalide
0	tout	0	valide(mine)
tout	0	message d'erreur	invalide(qte)
1	1	1	valide
1	2	1	valide
1	3	1	valide
1	4	message d'erreur	invalide
99	1	1	valide
99	2	2	valide
99	3	3	valide
99	4	message d'erreur	invalide
100	1	1	valide
100	2	2	valide
100	3	3	valide
100	4	message d'erreur	invalide

101	1	message d'erreur	invalide
101	2	message d'erreur	invalide
101	3	message d'erreur	invalide
101	4	message d'erreur	invalide

Tests junits

```
package TEST;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.example.Model.Mine;
import org.example.Model.TypeMinerai;



public class TestExtraireMine {
    protected Mine mine;

    @BeforeEach
    void setUp() throws Exception {
        mine = new Mine(1, null, TypeMinerai.OR,
50);
    }

    @AfterEach
    public void tearDown() throws Exception {
        mine = null;
    }
    @Test
    void test1() {
        mine = new Mine(1, null, TypeMinerai.OR,
0);
```

```
        assertEquals(0, mine.extraire(2));
    }
    @Test
    void test2() {
        mine = new Mine(1, null, TypeMinerai.OR,
1);
        assertEquals(1, mine.extraire(1));
    }
    @Test
    void test3() {
        mine = new Mine(1, null, TypeMinerai.OR,
1);
        assertEquals(1, mine.extraire(2));
    }
    @Test
    void test4() {
        mine = new Mine(1, null, TypeMinerai.OR,
1);
        assertEquals(1, mine.extraire(3));
    }
    @Test
    void test5() {
        mine = new Mine(1, null, TypeMinerai.OR,
99);
        assertEquals(1, mine.extraire(1));
    }
    @Test
    void test6() {
        mine = new Mine(1, null, TypeMinerai.OR,
99);
        assertEquals(2, mine.extraire(2));
    }
    @Test
    void test7() {
```

```
        mine = new Mine(1, null, TypeMinerai.OR,
99);
        assertEquals(3, mine.extraire(3));
    }

    @Test
    void test8() {
        mine = new Mine(1, null, TypeMinerai.OR,
100);
        assertEquals(1, mine.extraire(1));
    }

    @Test
    void test9() {
        mine = new Mine(1, null, TypeMinerai.OR,
100);
        assertEquals(2, mine.extraire(2));
    }

    @Test
    void test10() {
        mine = new Mine(1, null, TypeMinerai.OR,
100);
        assertEquals(3, mine.extraire(3));
    }
    @Test
    void test11() {
        assertThrows(IllegalStateException.class,
() -> {
            mine = new Mine(1, null,
TypeMinerai.OR, -1);
            mine.extraire(2);
        });
    }
```

```
    @Test
    void test12() {

assertThrows(IllegalArgumentException.class, () ->
{
            mine.extraire(-1);
        });
    }

    @Test
    void test14() {

assertThrows(IllegalArgumentException.class, () ->
{
            mine.extraire(0);
        });
    }

    @Test
    void test15() {
        mine = new Mine(1, null, TypeMinerai.OR,
1);

assertThrows(IllegalArgumentException.class, () ->
{
            mine.extraire(4);
        });
    }

    @Test
    void test16() {
        mine = new Mine(1, null, TypeMinerai.OR,
99);
```

```
assertThrows(IllegalArgumentException.class, () ->
{
            mine.extraire(4);
        });
    }

    @Test
    void test17() {
        mine = new Mine(1, null, TypeMinerai.OR,
100);

assertThrows(IllegalArgumentException.class, () ->
{
            mine.extraire(4);
        });
    }

    @Test
    void test18() {
        assertThrows(IllegalStateException.class,
() -> {
            mine = new Mine(1, null,
TypeMinerai.OR, 101);
            mine.extraire(1);
        });
    }

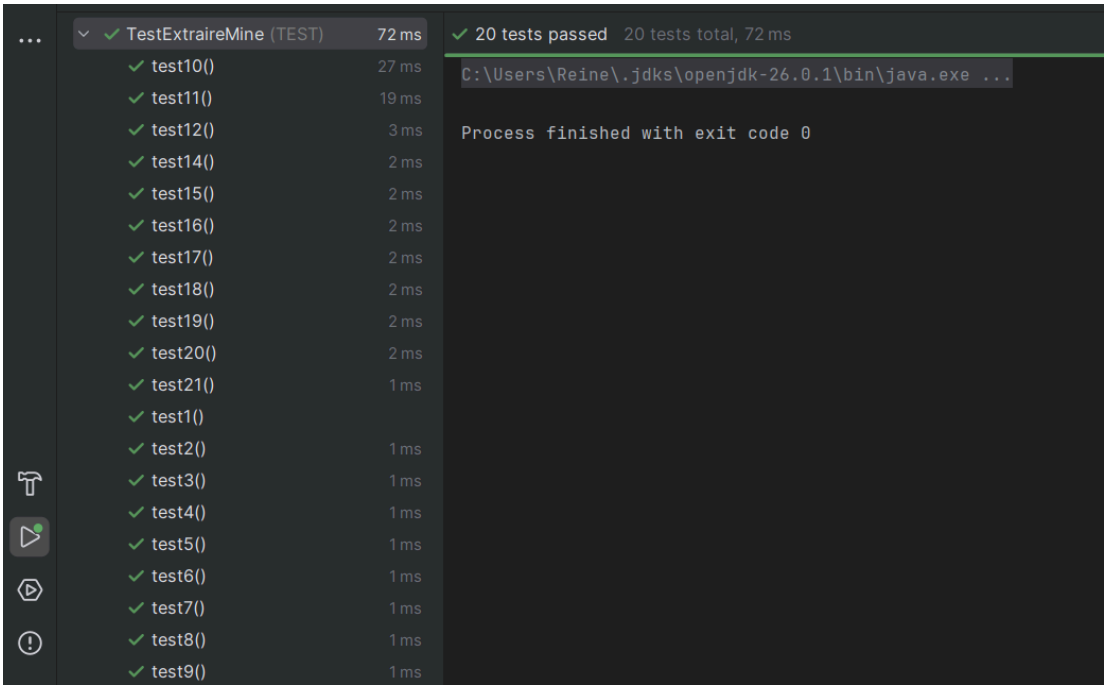
    @Test
    void test19() {
        assertThrows(IllegalStateException.class,
() -> {
            mine = new Mine(1, null,
TypeMinerai.OR, 101);
            mine.extraire(2);
```

```
        });
    }

    @Test
    void test20() {
        assertThrows(IllegalStateException.class,
() -> {
            mine = new Mine(1, null,
TypeMinerai.OR, 101);
            mine.extraire(3);
        });
    }

    @Test
    void test21() {
        assertThrows(IllegalStateException.class,
() -> {
            mine = new Mine(1, null,
TypeMinerai.OR, 101);
            mine.extraire(4);
        });
    }
}
```

Exécution des Tests / Méthodes extraire


TestExtraireMine (TEST) 72 ms
test10() 27 ms
test11() 19 ms
test12() 3 ms
test14() 2 ms
test15() 2 ms
test16() 2 ms
test17() 2 ms
test18() 2 ms
test19() 2 ms
test20() 2 ms
test21() 1 ms
test1()
test2() 1 ms
test3() 1 ms
test4() 1 ms
test5() 1 ms
test6() 1 ms
test7() 1 ms
test8() 1 ms
test9() 1 ms
20 tests passed 20 tests total, 72 ms
C:\Users\Reine\.jdks\openjdk-26.0.1\bin\java.exe ...
Process finished with exit code 0

class Robot : méthode deposer

Tableau Boite blanche class Robot : méthode deposer

Code de la méthode déposer:

```
119     public boolean deposer() {  6 usages   reinesorgho +2
120         Composantes c = secteur.getComposante();
121
122         if (!(c instanceof Entrepot)) {
123             System.out.println("Action invalide : pas d'entrepot sur ce secteur !");
124             return false;
125         }
126
127         Entrepot entrepot = (Entrepot) c;
128
129         if (entrepot.getType() != type) {
130             System.out.println("Action invalide : ce n'est pas le bon entrepot !");
131             return false;
132         }
133         if (nbMinerais == 0) {
134             System.out.println("Action invalide : le robot ne porte rien !");
135             return false;
136         }
137
138         entrepot.deposer(nbMinerais);
139         System.out.println("Robot " + numero + " a depose " + nbMinerais + " minerai(s).");
140         nbMinerais = 0;
141         return true;
```

Tableau de Test :

Instructions:

chemin	Composante	Robot	Type Minerais	nb Minerais	Résultat attendu
(109-112-115-132)	Mine	-	-	-	false
(109-112-119-122-132)	Entrepot	or	nickel	-	false
(109-112-119-123-126-132)	Entrepot	or	or	0	false
(109-112-119-123-131-132)	Entrepot	nickel	nickel	3	true

Les tests Junits :

```
package TEST;
```

```java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import org.example.Model.Robot;
import org.example.Model.Secteur;
import org.example.Model.Mine;
import org.example.Model.Entrepot;
import org.example.Model.TypeMinerai;

public class TestDeposerRobot {
    protected Secteur secteur;
    protected Robot robot;

    @BeforeEach
    void setUp() throws Exception {

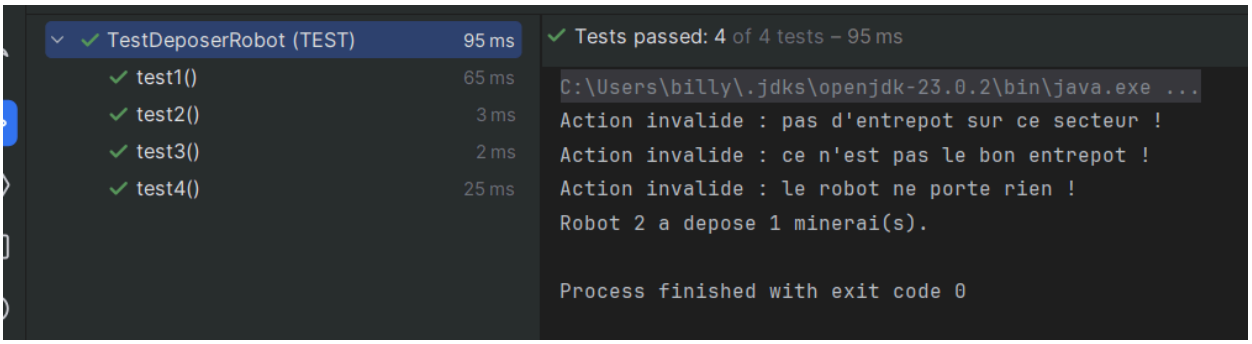
        secteur = new Secteur(1,1,false);
    }
    @AfterEach
    public void tearDown() throws Exception {
        secteur = null;
        robot = null;
    }
    @Test
    void test1()
    {
        secteur.setComposante(new Mine(1, secteur,
TypeMinerai.OR, 50));
        robot = new
Robot(2,secteur,TypeMinerai.OR,55,2);
        assertEquals(false, robot.deposer());
    }
    @Test
```

```
    void test2()
    {
secteur.setComposante(new
Entrepot(TypeMinerai.NICKEL,1,secteur));
        robot = new
Robot(2,secteur,TypeMinerai.OR,55,2);
        assertEquals(false, robot.deposer());

    }
    @Test
    void test3()
    {
        secteur.setComposante(new
Entrepot(TypeMinerai.OR,1,secteur));
        robot = new
Robot(2,secteur,TypeMinerai.OR,55,0);
        assertEquals(false, robot.deposer());

    }
    @Test
    void test4()
    {
        secteur.setComposante(new
Entrepot(TypeMinerai.OR,2,secteur));
        robot = new
Robot(2,secteur,TypeMinerai.OR,55,1);
        robot.SetNbMinerais(1);
        assertEquals(true, robot.deposer());
    }
}
```

Exécution des Tests / Méthodes déposer



Conclusion

Conclusion générale

Ce projet de tests automatisés réalisé dans le cadre de cette SAÉ, nous a permis de mettre en pratique les notions fondamentales du test logiciel avec JUnit 5, en combinant les approches boîte blanche et boîte noire. Grâce aux différents scénarios élaborés, nous avons pu vérifier le bon fonctionnement des principales méthodes de la classe Robot, notamment les déplacements (avancer), l'extraction de minerais (extraire) et le dépôt des ressources (déposer).

Les tests réalisés ont permis de valider les règles métiers imposées par le sujet, telles que la gestion des obstacles, le respect des limites d'extraction, la prise en compte des types de minerais et la vérification des conditions nécessaires au dépôt dans les entrepôts. L'analyse des résultats d'exécution et de la couverture de code montre que l'application répond correctement aux comportements attendus et qu'elle possède un niveau satisfaisant de fiabilité et de robustesse.

Ce travail nous a également permis de comprendre l'importance des tests automatisés dans le développement logiciel. En effet, ils facilitent la détection des erreurs, limitent les régressions et garantissent une meilleure qualité du code. Cette expérience a renforcé nos compétences en programmation Java, en conception de scénarios de tests et en utilisation des outils professionnels de développement logiciel.